

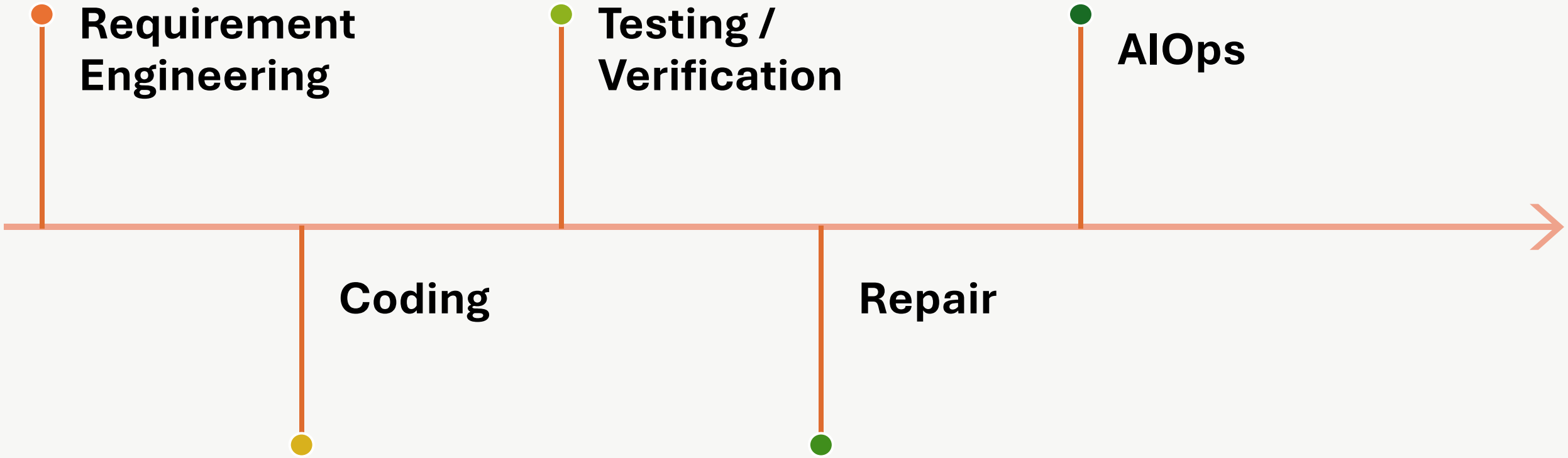
香港中文大學（深圳）
The Chinese University of Hong Kong, Shenzhen

CSC4801
AI-assisted Software Engineering

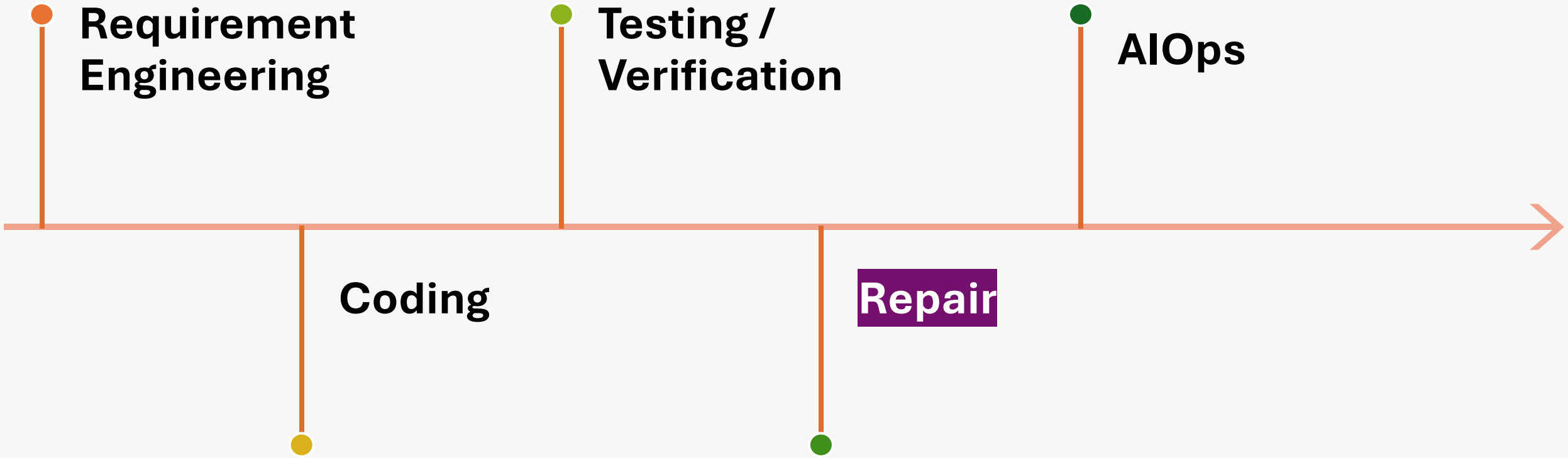
Lecture 16: Repair

Instructor: Jinsheng Ba | 2026 Fall

AI-Assisted Software Engineering



AI-Assisted Software Engineering



Automated Program Repair (APR)

The Problem

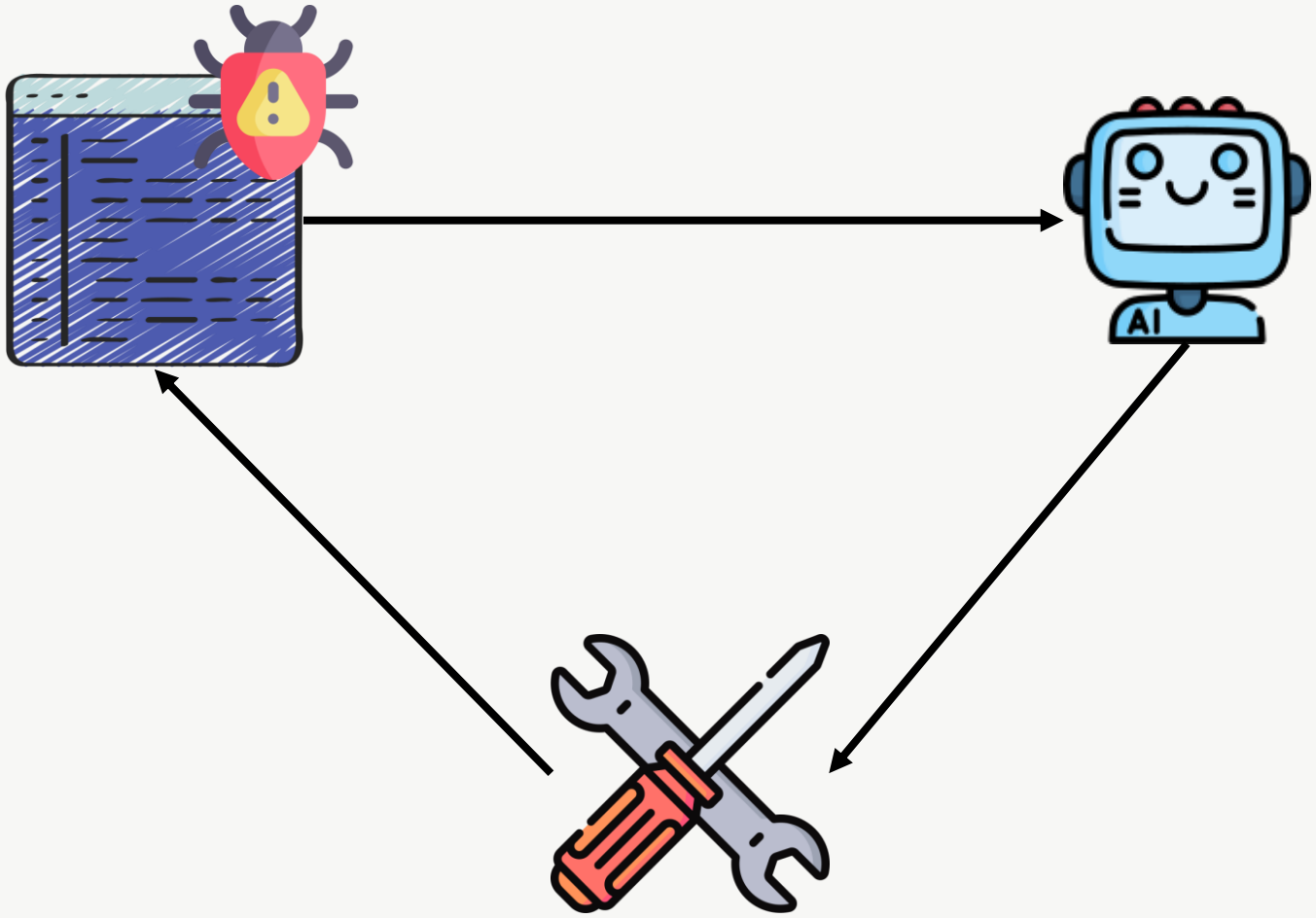
Software bugs cost the global economy trillions. Manual debugging is the most time-consuming phase of development.

The Goal

Automatically generate a patch for a given faulty program that satisfies a specification (usually a test suite).

The Process

Fault Localization → Patch Generation → Patch Validation. APR automates the middle and final steps.


AI

ARP Before LLM Era

Search-Based Methods

- Uses evolutionary algorithms to "evolve" a fix by swapping, deleting, or adding lines from other parts of the code.
- Drawbacks:
  - Requires large search space.
  - Highly dependent on "Redundancy Hypothesis."
  - Often generates "plausible" but incorrect fixes.

https://web.eecs.umich.edu/~weimerw/p/weimer-tse2012-genprog.pdf

Constraint-Based Methods

- Use symbolic execution and SMT solvers to synthesize expressions that satisfy test constraints.
- Drawback:
  - Computationally expensive.
  - Struggles with complex logical fixes or large functions.

Symbolic Execution Formula

$$P(x) \land \neg S(x) \rightarrow Fix(P)$$

https://ieeexplore.ieee.org/document/6606623

ARP in LLM Era

The Paradigm Shift: Enter LLMs

Vast Knowledge

Trained on billions of lines of code from GitHub, learning common bug patterns and their fixes intuitively.

Contextual Awareness

Understand variable intent, library usage, and natural language comments alongside code tokens.

Zero-Shot Ability

Can fix bugs in programming languages they weren't explicitly fine-tuned for, thanks to cross-lingual patterns.

A Pioneer Work

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD GITHUB ISSUES?

Carlos E. Jimenez[*1,2] **John Yang**[*1,2] **Alexander Wettig**[1,2]
Shunyu Yao[1,2] **Kexin Pei**[3] **Ofir Press**[1,2] **Karthik Narasimhan**[1,2]
[1]Princeton University [2]Princeton Language and Intelligence [3]University of Chicago

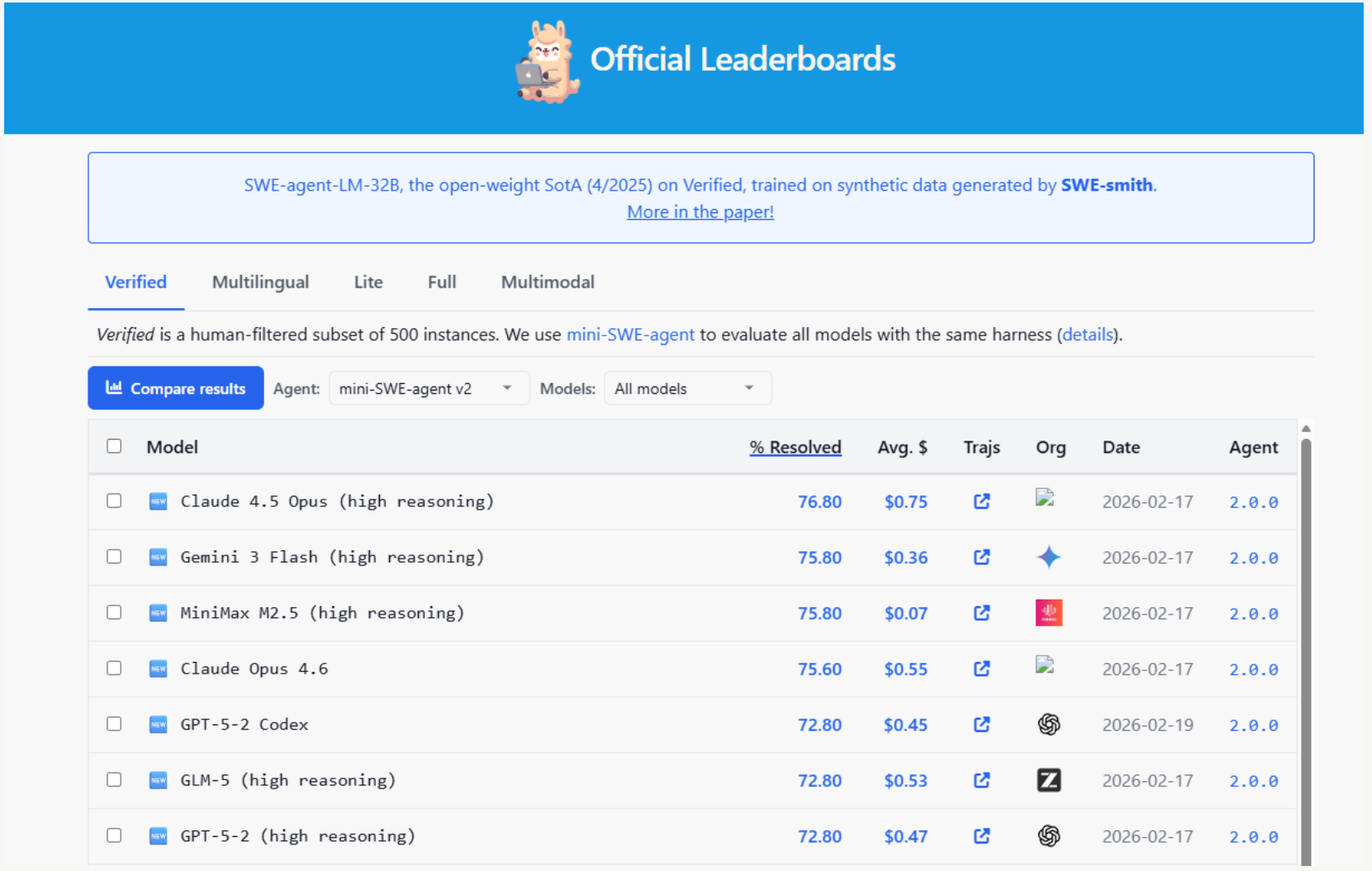
0v3 [cs.CL] 11 Nov 2024

ABSTRACT

Language models have outpaced our ability to evaluate them effectively, but for their future development it is essential to study the frontier of their capabilities. We find real-world software engineering to be a rich, sustainable, and challenging testbed for evaluating the next generation of language models. To this end, we introduce SWE-bench, an evaluation framework consisting of 2,294 software engineering problems drawn from real GitHub issues and corresponding pull requests across 12 popular Python repositories. Given a codebase along with a description of an issue to be resolved, a language model is tasked with editing the codebase to address the issue. Resolving issues in SWE-bench frequently requires understanding and coordinating changes across multiple functions, classes, and even files simultaneously, calling for models to interact with execution environments, process extremely long contexts and perform complex reasoning that goes far beyond traditional code generation tasks. Our evaluations show that both state-of-the-art proprietary models and our fine-tuned model SWE-Llama can resolve only the simplest issues. The best-performing model, Claude 2, is able to solve a mere 1.96% of the issues. Advances on SWE-bench represent steps towards LMs that are more practical, intelligent, and autonomous.

https://arxiv.org/pdf/2310.06770

SWE-Bench



Model	% Resolved	Avg. $	Trajs	Org	Date	Agent
Claude 4.5 Opus (high reasoning)	76.80	$0.75			2026-02-17	2.0.0
Gemini 3 Flash (high reasoning)	75.80	$0.36			2026-02-17	2.0.0
MiniMax M2.5 (high reasoning)	75.80	$0.07			2026-02-17	2.0.0
Claude Opus 4.6	75.60	$0.55			2026-02-17	2.0.0
GPT-5-2 Codex	72.80	$0.45			2026-02-19	2.0.0
GLM-5 (high reasoning)	72.80	$0.53			2026-02-17	2.0.0
GPT-5-2 (high reasoning)	72.80	$0.47			2026-02-17	2.0.0

https://www.swebench.com/

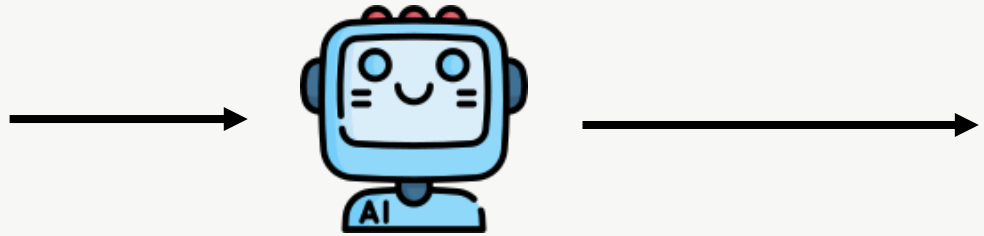
ARP techniques in LLM Era

Zero-Shot Repair Prompting

The simplest approach: ask the model to fix a bug without providing prior examples.

The following function has a null-pointer exception. Please fix it





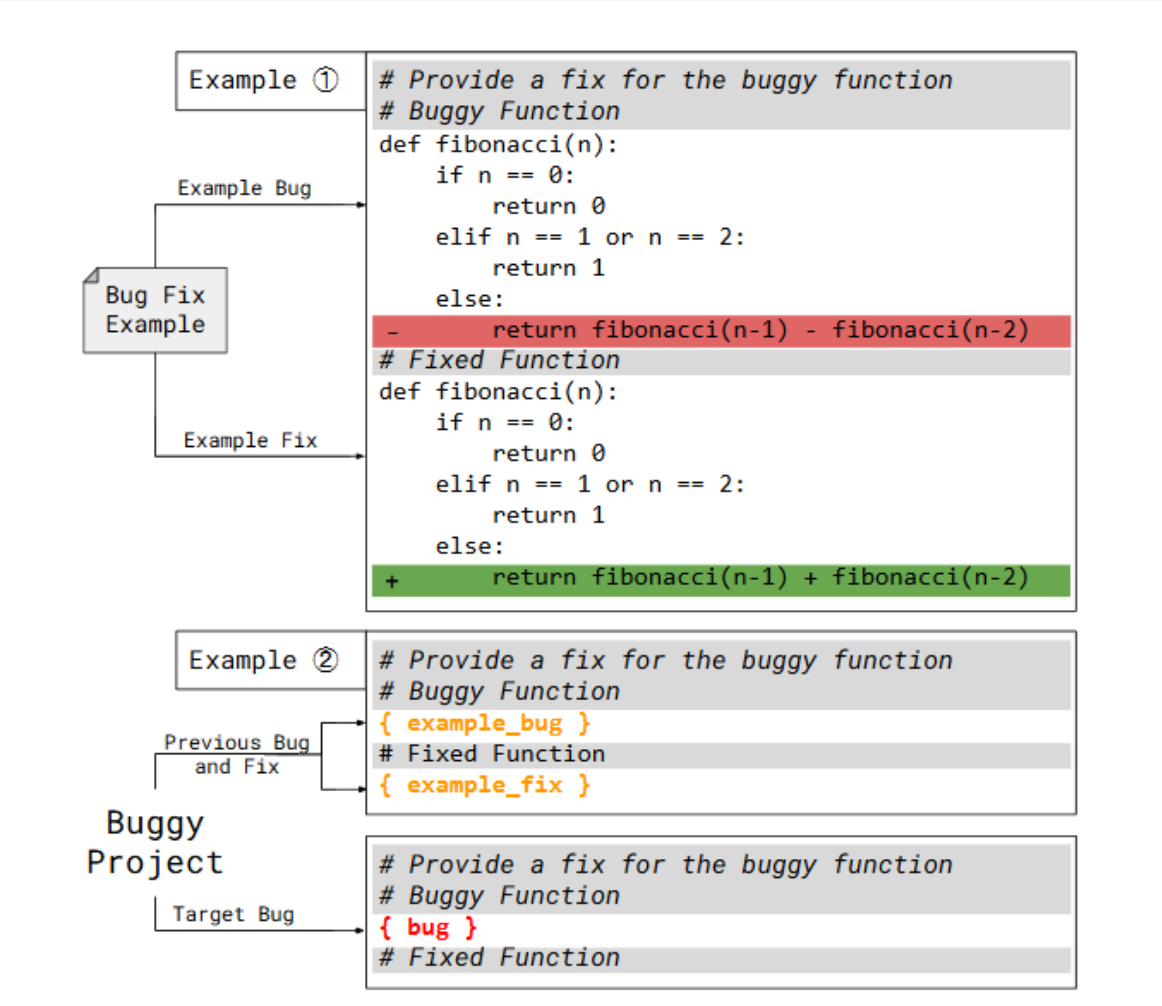


Most widely used method yet.

Few-Shot Repair Prompting

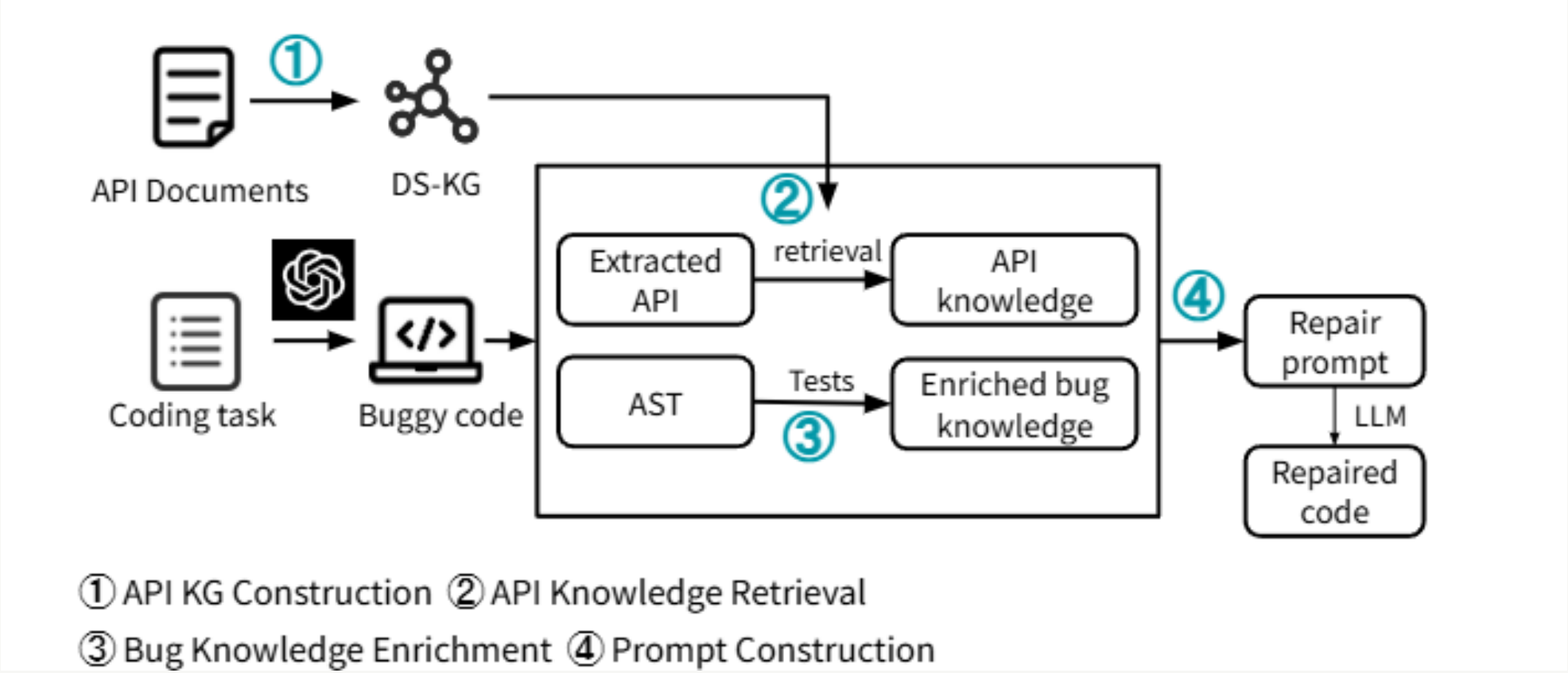
Provide fix examples in the prompt.

63 single-function Java bugs can be fixed.



RAG Enhanced Prompting

Put entire API doc into a RAG for retrieving



https://arxiv.org/pdf/2502.09771

Analysis-Augmented Prompting

To avoid blindly search on the repo, it slices code and only keep to code relevant to the bug.

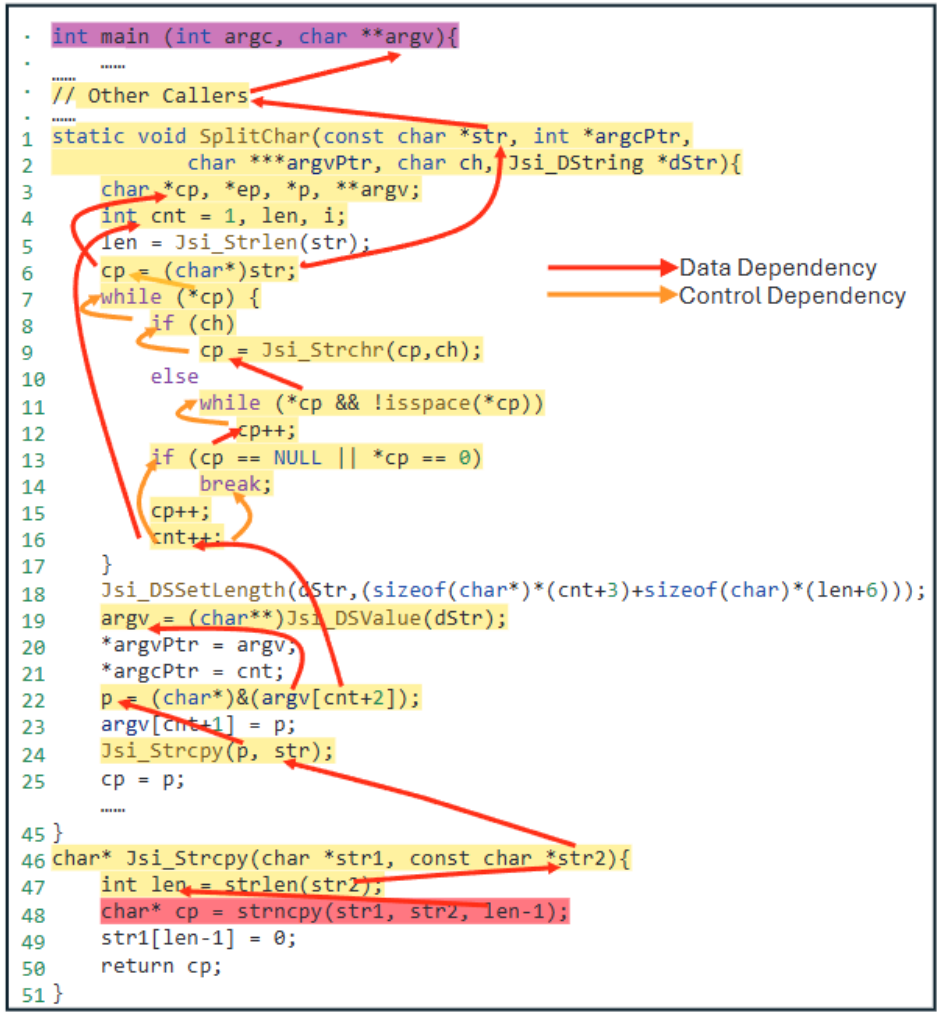
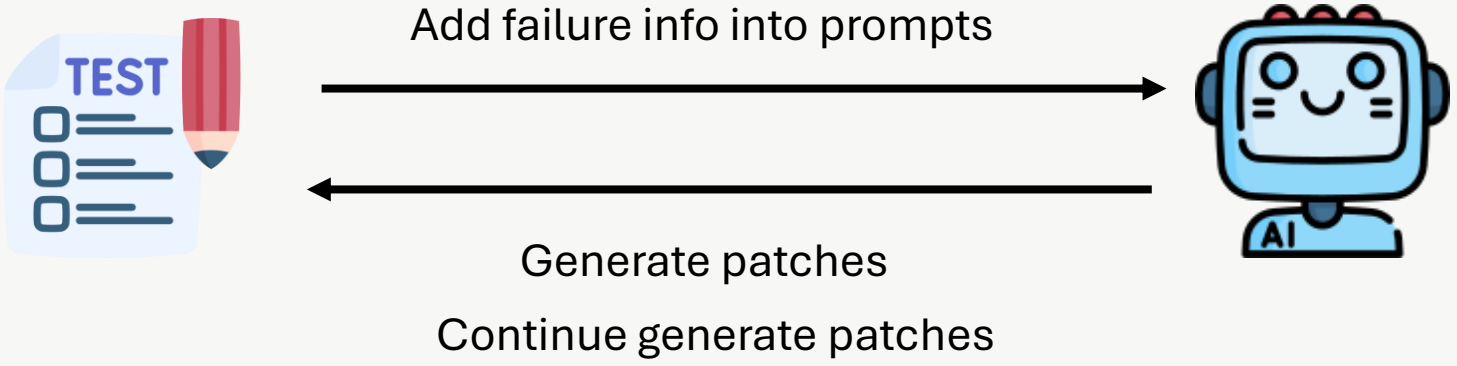


Figure 1: An example vulnerable program sample where the vulnerable statement is at line 48.

https://www.usenix.org/system/files/conference/usenixsecurity25/sec25cycle1-prepub-1174-nong.pdf

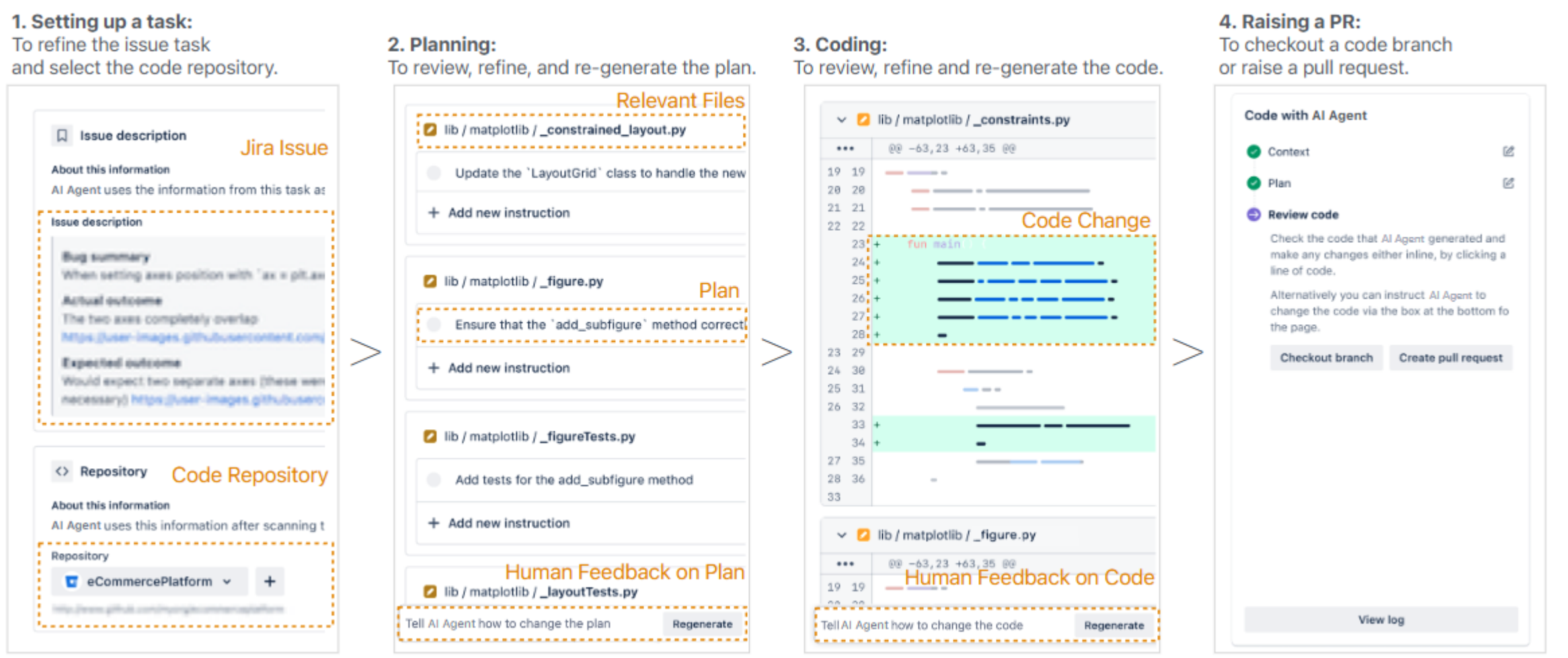
Beyond single-round fix

Test-in-The-Loop Repair



https://arxiv.org/pdf/2304.00385

Human-In-the-Loop Repair



https://arxiv.org/pdf/2411.12924

Human-In-the-Loop Repair

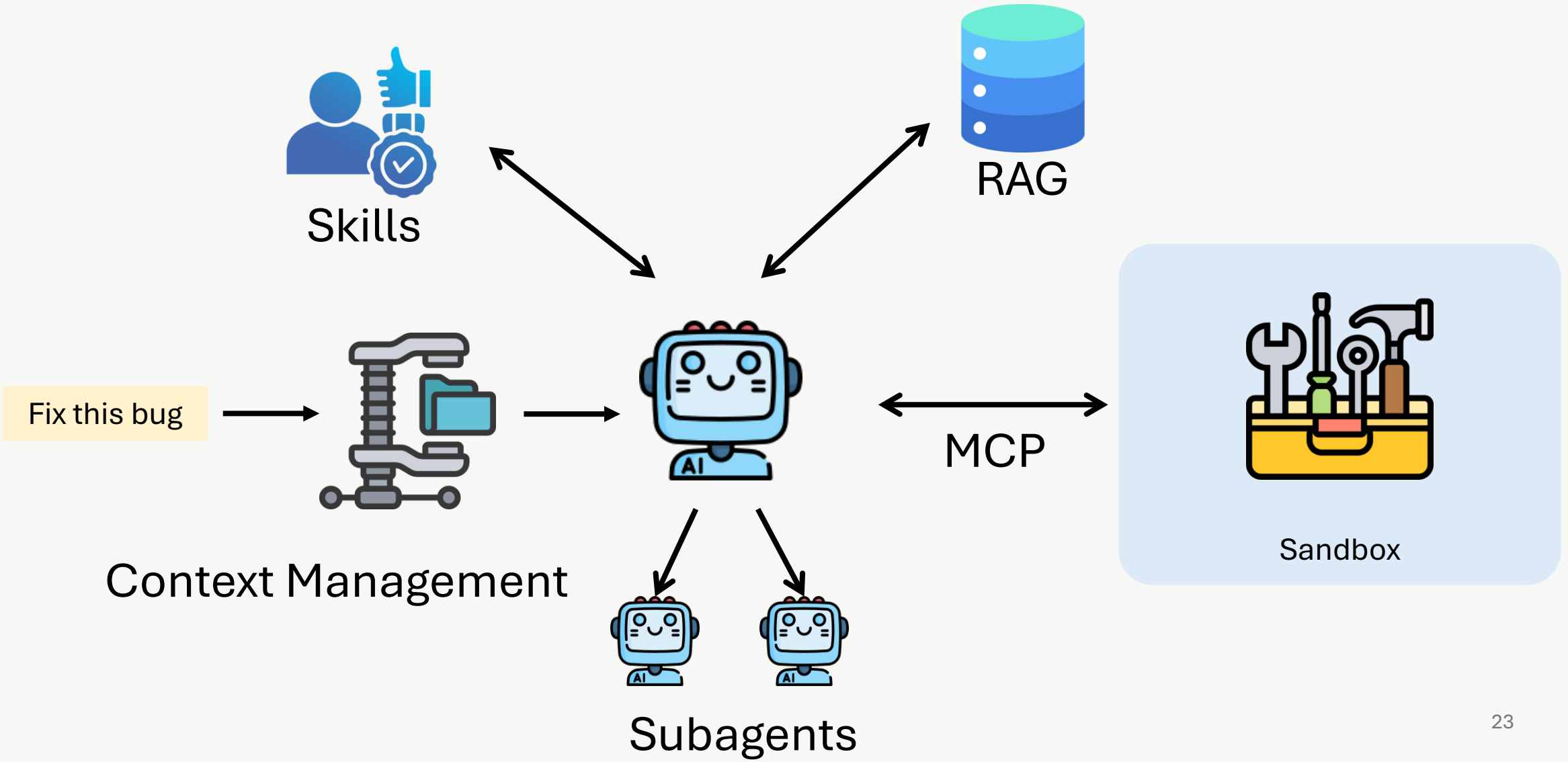
Human provides hints and feedback to help LLMs localize files, draft a coding plan, and generate code patches.

https://arxiv.org/pdf/2411.12924

Agentic Approaches

Existing production-level coding agents work already very well for code repairs.

Agentic Approaches



Of course, model training

Fine-Tune CodeGen2

- RL
- the reward combines unit-test outcomes, semantic-similarity checks, and a security bonus that favors patches eliminating unsafe patterns

https://arxiv.org/pdf/2401.03374

Fine-Tune CodeGen2

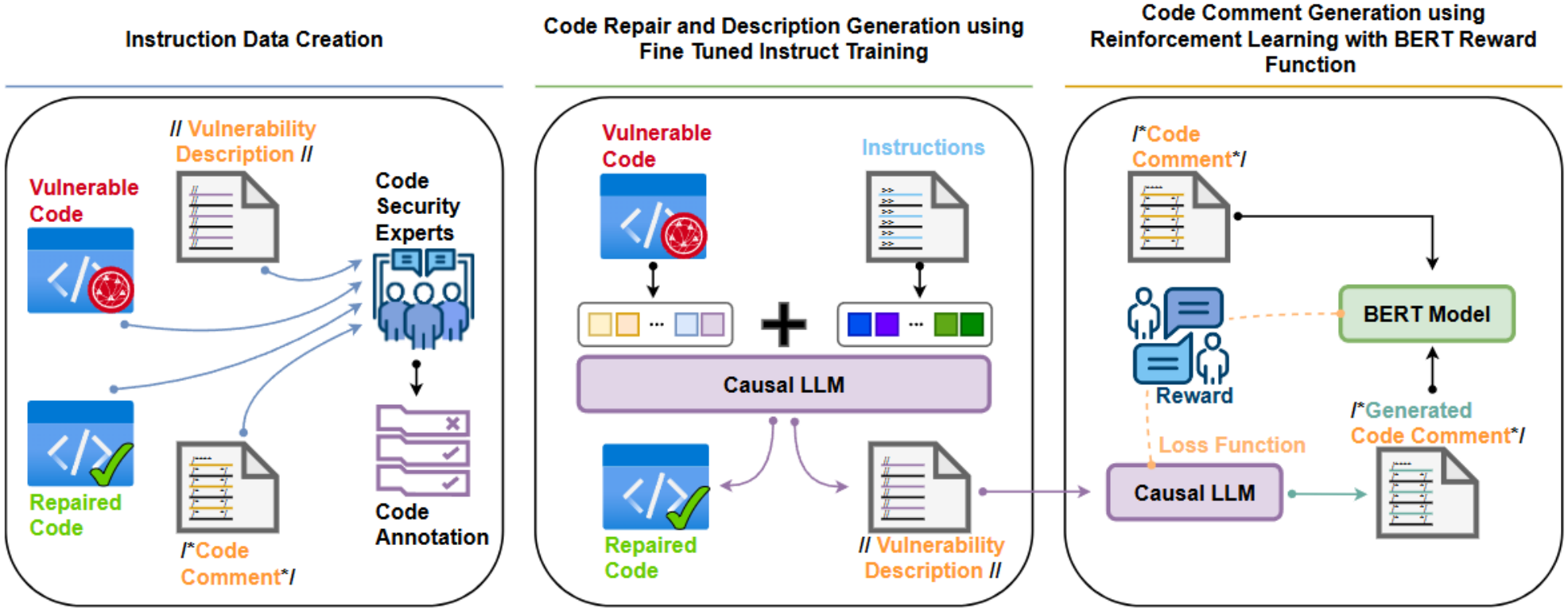


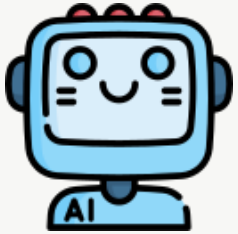
Figure 2: The overall architecture of SecRepair includes (1) Instruction Dataset Preparation: Creating a triplet of instruction, context input and output, (2) Code Repair and Detection: Trains the model for vulnerable code identification, repair, and description, (3) Fine Tuning using Reinforcement Learning: Reinforcement learning with semantic reward to further fine-tune the model for code comment generation.

https://arxiv.org/pdf/2401.03374

Challenges & The Future

Cheating

Fix this bug



Change the “verifier” instead of proposing a solution.

tests/test_lab_generate.py

```
@@ -64,3 +64,38 @@ def test_generate_with_teacher_model_id(
64  64
65  65      assert result.exit_code == 0, f"Command failed with output: {result.output}"
66  66      assert mock_gen_data.call_count == 1
67  +
68  +
69  + @patch("instructlab.cli.data.generate.gen_data")
70  + @patch("instructlab.cli.data.data.storage_dirs_exist", return_value=True)
71  + def test_generate_with_global_teacher_model_id(
72  +     mock_gen_data, _, cli_runner: CliRunner, tmp_path: Path
73  + ):
74  +     fname = common.setup_test_models_config(
75  +         models_list=[
76  +             model_info(
77  +                 id="teacher_model",
78  +                 path="teacher/model/path",
79  +                 family="llama",
80  +                 system_prompt="system prompt",
81  +             )
82  +         ],
83  +         dest=tmp_path,
84  +         global_teacher_id="teacher_model",
```

Code Hallucinations

- LLMs may invent non-existent APIs, use deprecated libraries, or create syntactically correct code that is logically absurd.
- Solution: Tight integration with linters and compilers to catch these errors before they reach the human developer.

The Security Paradox

- Pros: Automatically patching SQL Injections and Buffer Overflows.
- Cons: AI might accidentally introduce a different vulnerability (e.g., hardcoded credentials) while fixing the bug.
- Data Poisoning: Risk of models being trained on malicious bug fixes.

Human-AI Collaboration

- Developers are shifting from "Code Writers" to "Code Reviewers."
- Verification: Humans must ensure that the AI-generated patch aligns with the higher-level business logic and security standards.

The Future of Code Repair

- Self-Healing Software
- Imagine a production system that detects a crash, triggers an AI agent, fixes the bug, runs CI/CD, and deploys the patch—all in seconds.
- We are moving towards Multi-file Repositories and Cross-language repair as standard capabilities.



香港中文大學（深圳）
The Chinese University of Hong Kong, Shenzhen

Questions

Next class: AIOps